

Design a Notification System

Instagram · Slack · Swiggy · ప్రతి app – "ఒక సంఘటన జరిగింది, వాడుకరికి చెప్పేద్దాం" అని అందరూ మొదలుపెడతారు.

మూడు విరుపులు. మొదటిది – ఒక వైరల్ post ఒక్క వ్యక్తి ఫోన్‌కి **50,000 notifications** పంపుతుంది. రెండోది – పరిమితి లేకపోతే **78.4% వాడుకరులు** notifications పూర్తిగా ఆపేస్తారు, మరియు వాళ్ళు శాశ్వతంగా పోతారు. మూడోది – మొత్తంలో **5% ఉన్న SMS** మిగతా 95% ని **17 సెకన్లు** ఆపేస్తుంది.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి శాతం నిజంగా `node` lo run చేసినదే.

RETRY, BACKOFF, DLQ గురించి ఇక్కడ ఒక్క విరుపు కూడా లేదు – అవి LLD DEEP 25 LO ఉన్నాయి

Notification system అనగానే అందరూ **delivery** గురించి మాట్లాడతారు: retry, exponential backoff, dead letter queue. అవన్నీ ముఖ్యమే, మరియు వాటిని **LLD Deep 25 (Job Scheduler) §4 lo కొలిచాం** – jitter లేని backoff ఒక్క పిలుపు కూడా ఆదా చేయలేదు, మరియు 22 రెట్లు ఎక్కువ సమయం తీసుకుంది. ఆ కొలతలు ఇక్కడా అలాగే వర్తిస్తాయి.

కాబట్టి ఈ doc వేరే ప్రశ్న అడుగుతుంది, మరియు అది delivery గురించి కాదు:

"ఇది అసలు పంపాలా?"

మూడు విరుపులూ ఆ ఒక్క ప్రశ్న యొక్క మూడు రూపాలు – **ఇప్పుడే పంపాలా** (§4), **ఇంకా పంపాలా** (§7), **ఏ దారిలో పంపాలా** (§10).

మరియు §7 lo ఒక ఫలితం వస్తుంది, అది ఈ doc మొత్తానికి సారాంశం: **తక్కువ పంపితే ఎక్కువ చేరుతుంది.**

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు నిజమైన ప్రశ్న ఏమిటి?
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: వైరల్ అయిన క్షణం

3. Step – సంఘటన వచ్చింది, notification పంపు
4. మొదటి విరుపు – 4 గంటల్లో 50,000 notifications
5. Step – కిటికీలో కలపడం

Part 3 – రెండో విరుపు: వాడుకరి అలసిపోతాడు

6. Step – రోజుకి 243 notifications
7. రెండో విరుపు – 78.4% వాడుకరులు శాశ్వతంగా ఆపేస్తారు

Part 4 – మూడో విరుపు: నెమ్మదైన మార్గం

8. Step – push · email · SMS · ఒకే queue
9. మూడో విరుపు – 5% traffic, 95% ఆలస్యం
10. Step – ఒక్క మార్గానికి సొంత pool

Part 5 – పూర్తి system

Part 6 – Interview lo

12. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి
13. నోటీతో చెప్పాల్సిన English script
14. Follow-ups – templates, A/B, ఖర్చు
15. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు నిజమైన ప్రశ్న ఏమిటి?

ఒక సంఘటన జరుగుతుంది – ఎవరో like చేశారు, ఒక order బయలుదేరింది, ఒక payment విఫలమైంది. దాన్ని వాడుకరికి చెప్పాలి: push, email, లేదా SMS.

సాంకేతికంగా ఇది సులభం. ఒక queue, కొన్ని workers, మూడు provider APIs. రెండు రోజుల్లో రాయొచ్చు.

కానీ ఈ system యొక్క నిజమైన కష్టం **సాంకేతికం కాదు**, మరియు అది ఒక్క ప్రశ్నలో ఉంది:

NOTIFICATION SYSTEM యొక్క అసలు ప్రశ్న

"ఈ సంఘటనని ఈ వ్యక్తికి, ఈ క్షణంలో, ఈ మార్గంలో చెప్పడం విలువైనదేనా?"

మిగతా అన్ని systems lo – database, cache, queue – "ఎక్కువ throughput మంచిది". ఇక్కడ కాదు.

ఇక్కడ throughput పెంచడం మీ వాడుకరులని **శాశ్వతంగా** పొగొడుతుంది (§7). మరియు ఆ నష్టాన్ని మీరు ఎప్పటికీ తిరిగి పొందలేరు – ఒకసారి notifications ఆపేసిన వాడుకరి మళ్ళీ ఆన్ చేయడు.

అనుకుందాం: 5 లక్షల వాడుకరులు · రోజుకి సగటున 243 సంఘటనలు ఒక్కొక్కరికి · మూడు మార్గాలు.

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	ఎందుకు అడుగుతున్నాం
ఒక వాడుకరికి రోజుకి గరిష్టంగా ఎన్ని పంపొచ్చు?	ఇదే §7 మొత్తం. పరిమితి లేకపోతే 78.4% పోతారు

ప్రశ్న	ఎందుకు అడుగుతున్నాం
ఒకే రకం సంఘటనలు కలపాచ్చా?	"50 మంది like చేశారు" vs 50 notifications – §5
ఆలస్యం ఎంత ఫరవాలేదు?	30 సెకన్ల కిటికీ 98.2% తగ్గిస్తుంది. 0 సెకన్లు అంటే ఏమీ ఆదా కాదు
ఏ notifications తక్షణం కావాలి?	OTP, security alert – ఇవి కలపకూడదు. మిగతావన్నీ కలపొచ్చు
SMS ఖర్చు ఎంత?	SMS push కంటే వందల రెట్లు ఖరీదు – ఇది ఒక routing నిర్ణయం
Provider పరిమితులు ఏమిటి?	§9 – SMS provider సెకనుకి 10 అంటే అది మీ queue ని ఆపేస్తుంది
నిశ్శబ్ద గంటలు ఉన్నాయా?	ఉంటే ఆ notifications పారేయకూడదు, వాయిదా వేయాలి – §11 lo ఇది ఒక bug ని పట్టుకుంది

ఒక ప్రశ్న **ENGINEERING** ప్రశ్నలా అనిపించదు, కానీ అదే అతి ముఖ్యమైనది

"ఒక వాడుకరి notifications ఆపేస్తే, మనకి ఎంత నష్టం?"

ఎందుకంటే అది ఒక శాశ్వత, తిరిగి పొందలేని నష్టం. §7 lo కొలుస్తాం: పరిమితి లేకుండా 30 రోజుల్లో **3,92,130 వాడుకరులు** ఆపేస్తారు – అంటే మీరు వాళ్ళకి ఇక ఎప్పటికీ ఏమీ పంపలేరు.

Interview lo ఇది అడిగితే, మీరు **system ని కాదు, product ని** ఆలోచిస్తున్నారని తెలుస్తుంది.

Part 2 – మొదటి విరుపు: వైరల్ అయిన క్షణం

3. Step – సంఘటన వచ్చింది, notification పంపు

సహజమైన design. ఒక సంఘటన → ఒక notification:

```
function onEvent(event) {  
  const user = event.targetUser;  
  queue.push({ user, text: render(event), channel: 'push' });  
}
```

ఇది సరిగ్గా పని చేస్తుంది, మరియు 99% సమయం ఇది **సరైనది** కూడా. ఒక like వస్తే ఒక notification.

సమస్య మిగతా 1% lo – ఒక post వైరల్ అయినప్పుడు.

4. మొదటి విరుపు – 4 గంటల్లో 50,000 notifications

ఒక వైరల్ post • 4 గంటల్లో 50,000 likes • ఆ వ్యక్తి ఫోన్‌కి ఎన్ని notifications?

కలిపే కిటికీ	notifications	ఒక్క నిమిషానికి సగటు	తగ్గింపు
కలపకుండా	50,000	208.3	–
30 సెకన్లు	912	3.8	98.2%
5 నిమిషాలు	129	0.5	99.7%
1 గంట	14	0.1	100.0%

నిమిషానికి 208 NOTIFICATIONS – అంటే ప్రతి 0.3 సెకన్లకి ఒకటి

ఆ ఫోన్ వాడటం **అసాధ్యం**. మరియు ఇది ఒక bug కాదు – మీ code సరిగ్గానే పని చేస్తోంది. ప్రతి like కీ ఒక notification, సరిగ్గా అడిగినట్టు.

కానీ వాడుకరికి కావాల్సినది 50,000 సందేశాలు కాదు – అతనికి కావాల్సినది **ఒక్క వాక్యం**: "మీ post ని 50,000 మంది ఇష్టపడ్డారు."

సంఘటనలు మరియు notifications ఒకటి కాదు. ఆ రెండింటినీ ఒకటే అనుకోవడమే §3 code యొక్క తప్పు.

5. Step – కిటికీలో కలపడం

పరిష్కారం: సంఘటనని వెంటనే పంపకుండా, ఒక కిటికీ పాటు పోగు చేసి, తర్వాత ఒక్క notification పంపడం.

```
// ఒక సంఘటన వచ్చింది. దీన్ని *వెంటనే* పంపము - §4 lo చూసినట్టు కలుపుతాం.
notify(user, kind, payload) {
  this.stats.received++;
  ...
  const key = `${user}|${kind}`;
  let b = this.#pending.get(key);
  if (!b) {
    b = { user, kind, events: [], firstAt: this.#clock() };
    this.#pending.set(key, b);
  }
  else this.stats.coalesced++; // ఇది ఇప్పటికే ఉన్నదానిలో కలిసిపోతోంది
  b.events.push(payload);
  return { ok: true, coalescedInto: b.events.length };
}
```

గమనించండి – key `user|kind`. అంటే likes కలుస్తాయి, comments కలుస్తాయి, కానీ **likes మరియు comments కలవవు**. ఆ రెండూ వేర్వేరు సందేశాలు.

30 సెకన్ల కిటికీ – 98.2% తగ్గింపు, మరియు వాడుకరికి మెరుగైన అనుభవం

ఇది ఒక అరుదైన సందర్భం: ఖర్చు తగ్గింపు మరియు నాణ్యత పెరుగుదల ఒకే మార్పులో.

50,000 సందేశాలు = వాడుకరికి చెత్త. 912 సందేశాలు = ఇంకా చెత్త, కానీ తక్కువ. మరియు 5 నిమిషాల కిటికీతో **129** – అది నిజంగా చదవదగినది.

మరియు ఇక్కడ ఒక కీలకమైన వివరం: **కిటికీ పెంచడం వల్ల ఆలస్యం పెరగదు** – మొదటి notification ఇంకా కిటికీ ముగిసిన వెంటనే వెళ్తుంది. పెరిగేది **మిగతావాటి** ఆలస్యం, మరియు అవి ఎలాగూ కలిసిపోతున్నాయి.

కానీ అన్నిటిని కలవకూడదు

OTP, security alert, payment విఫలం – ఇవి **తక్షణం** వెళ్ళాలి, మరియు ఇవి కలవకూడదు.

కాబట్టి `coalesceMs` ఒక **global setting** కాదు – అది ఒక్కో రకానికి ఒకటి. ఇది HLD Deep 01 §11 lo `CatchUp` విధానం లాంటిదే: **వ్యవస్థకి తెలియని నిర్ణయాన్ని వ్యవస్థ తీసుకోకూడదు.**

Part 3 – రెండో విరుపు: వాడుకరి అలసిపోతాడు

6. Step – రోజుకి 243 notifications

కలపడం వైరల్ క్షణాలని పరిష్కరించింది. కానీ సాధారణ రోజు ఎలా ఉంటుంది?

ఒక సాధారణ వాడుకరి • రోజులో అన్ని మూలాల నుంచి

likes/comments	120
messages	45
follows	8
group activity	60
app updates	6
marketing	4

మొత్తం 243 / రోజు = ప్రతి 6 నిమిషాలకి ఒకటి

ప్రతి ఒక్కటి **సహేతుకమైనది**. ఏ ఒక్క team కూడా తప్పు చేయలేదు – likes team likes పంపుతోంది, messages team messages పంపుతోంది, marketing team రోజుకి 4 మాత్రమే పంపుతోంది.

కానీ వాడుకరికి కనిపించేది **మొత్తం**: ప్రతి 6 నిమిషాలకి ఒక notification.

7. రెండో విరుపు – 78.4% వాడుకరులు శాశ్వతంగా ఆపేస్తారు

వాడుకరికి ఒక పరిమితి ఉంది. అది దాటితే అతను ఒక్క notification ని ఆపడు – అతను **అన్నిటినీ** ఆపేస్తాడు. మరియు అది **శాశ్వతం**.

దాన్ని కొలుద్దాం. 5 లక్షల వాడుకరులు, 30 రోజులు:

500k users • 30 రోజులు • 30 దాటిన ప్రతి notification కి 0.4% ఆపేసే అవకాశం

రోజువారీ పరిమితి	పంపినవి	ఆపేసినవి	notifications ఆపేసినవాళ్ళు ఇంకా చేరగలిగేవాళ్ళు
పరిమితి లేదు	345.6 మి	0.0 మి	3,92,130 (78.4%) 1,07,870
100 / రోజు	331.9 మి	57.8 మి	3,51,375 (70.3%) 1,48,625
50 / రోజు	356.0 మి	125.4 మి	2,49,383 (49.9%) 2,50,617
30 / రోజు	399.7 మి	259.7 మి	0 (0.0%) 5,00,000

మొదటి మరియు నాలుగో వరుసని పక్కపక్కన చూడండి

పరిమితి లేకుండా: 345.6 మిలియన్ notifications పంపాం. 30 రోజుల చివరికి **3,92,130 వాడుకరులు (78.4%)** notifications ఆపేశారు. మిగిలినవాళ్ళు **1,07,870**.

రోజుకి 30 పరిమితితో: 399.7 మిలియన్ పంపాం – అంటే **15.7% ఎక్కువ**. మరియు **ఒక్కరూ** ఆపేయలేదు.

తక్కువ పంపాలని నిర్ణయించుకుని, ఎక్కువ పంపగలిగాం.

30 రోజుల తర్వాత – ఎంతమందికి ఇంకా చేరగలం?

పరిమితి లేదు

3,92,130 ఆపేశారు (78.4%)

1,07,870

రోజుకి 30

5,00,000 – అందరూ ఇంకా చేరగలిగేవాళ్ళే

మొత్తం పంపినవి

345.6 మి · పరిమితి లేదు

399.7 మి · రోజుకి 30

తక్కువ పంపాలని నిర్ణయించుకుని, **15.7% ఎక్కువ** పంపగలిగాం.

ఈ MODEL LO నేను ఊహించినవి – మరియు ఏది కొలత

"30 దాటేతే ప్రతి అదనపు notification కి 0.4% ఆపేసే అవకాశం" – ఆ **రెండు సంఖ్యలు నా ఊహ**. మీ product lo అవి వేరుగా ఉంటాయి, మరియు వాటిని మీ స్వంత డేటా నుంచి తీయాలి.

కానీ **ఆకారం** ఊహ కాదు: **సహనం ఒక పరిమిత వనరు, మరియు అది తిరిగి నిండదు**. ఆ ఆకారం ఉన్నంతవరకు, పరిమితి పెట్టడం ఎప్పుడూ ఎక్కువ చేరవేస్తుంది.

15 పరిమితి 225 మిలియన్ మాత్రమే పంపింది – అంటే **మరీ కఠినమైన పరిమితి కూడా నష్టం**. అత్యుత్తమ స్థానం మధ్యలో ఉంది, మరియు అది మీ డేటా చెప్పాలి.

Part 4 – మూడో విరుపు: నెమ్మదైన మార్గం

8. Step – push · email · SMS · ఒకే queue

మూడు మార్గాలు, మూడు వేర్వేరు స్వభావాలు:

మార్గం	వేగం	provider పరిమితి	ఖర్చు
push (FCM/APNS)	~50 ms	దాదాపు అపరిమితం	ఉచితం
email	~400 ms	ఎక్కువ	చాలా తక్కువ
SMS	~2,000 ms	సెకనుకి 10	వందల రెట్లు ఎక్కువ

సహజమైన design: ఒకే queue, ఒకే worker pool, ప్రతి job తన మార్గానికి వెళ్తుంది.

అది సరళంగా ఉంది. మరియు అది పని చేస్తుంది – **SMS లేనంతవరకు**.

9. మూడో విరుపు – 5% traffic, 95% ఆలస్యం

6,000 notifications · 80% push · 15% email · 5% SMS · 40 workers
SMS provider సెకనుకి 10 మాత్రమే అంగీకరిస్తుంది

అమరిక	push p50	push p99	email p99	SMS p99
ఒకే queue అందరికీ	7.7 s	16.8 s	17.2 s	19.0 s

PUSH NOTIFICATION P50 = 7.7 సెకన్లు

Push సాంకేతికంగా **50 మిల్లీసెకన్లు** పని. అది 7.7 సెకన్లు ఎందుకు పడుతోంది?

ఎందుకంటే మొత్తం traffic లో **5% మాత్రమే ఉన్న SMS** workers ని పట్టుకుని కూర్చుంది. ప్రతి SMS 2 సెకన్లు, మరియు provider సెకనుకి 10 మాత్రమే అంగీకరిస్తుంది – కాబట్టి అవి **ఎదురుచూస్తూ** ఉంటాయి, workers ని ఖాళీ చేయకుండా.

ఇది LLD Deep 26 §7 లో చూసిన అదే ఆకారం: **ఒక నెమ్మదైన పని రకం మిగతా అన్నిటినీ ఆకలితో చంపుతుంది.**

10. Step – ఒక్క మార్గానికి సొంత pool

అమరిక	push p50	push p99	email p99	SMS p99
ఒకే queue అందరికీ	7.7 s	16.8 s	17.2 s	19.0 s
ఒక్క మార్గానికి సొంత pool	50 ms	50 ms	17.0 s	130.3 s

PUSH P99: 16.8 సెకన్లు → 50 మిల్లీసెకన్లు • 336 రెట్లు

Push ఇప్పుడు **ఖచ్చితంగా** 50 ms – ఎందుకంటే అది ఇక ఎవరి కోసమూ ఎదురుచూడటం లేదు.

ఇదే **bulkhead**: ఒక రకమైన పని ఇంకొక రకాన్ని ముట్టుకోకూడదు.

కానీ చివరి నిలుపు వరుస చూడండి – **SMS 19.0 s నుంచి 130.3 s కి పెరిగింది**. అది నిజాయితీగా చెప్పాల్సిన విషయం.

SMS ఎందుకు ఫోరం అయింది – మరియు అది ఎందుకు సరైనది

Separate pools lo SMS కి 40 lo 4 workers మాత్రమే. కాబట్టి దాని queue నెమ్మదిగా ఖాళీ అవుతుంది.

కానీ ఒక్క క్షణం ఆలోచించండి: **SMS provider సెకనుకి 10 మాత్రమే అంగీకరిస్తాడు**. మీరు 40 workers ఇచ్చినా అతను వేగంగా తీసుకోడు. ఆ queue **ఎలాగూ** నెమ్మదిగా ఖాళీ అవుతుంది.

కాబట్టి ప్రశ్న "SMS వేగంగా వెళ్తుందా" కాదు – అది **"SMS తన నెమ్మదితనాన్ని push మీదికి రుద్దుతుందా"**.

ఒక queue యొక్క వేగాన్ని దాని అతి నెమ్మదైన వినియోగదారు నిర్ణయించకూడదు.

Part 5 – పూర్తి system

11. మొత్తం code • 7.5 లక్షల సంఘటనలు • mutation testing

ఐదు నియమాలు

#	నియమం	విరిగితే అర్థం
1	రోజువారీ పరిమితి దాటకూడదు	§7 రక్షణ లేదు
2	ఆపేసిన వాడుకరికి ఏమీ వెళ్ళకూడదు	అనుమతి ఉల్లంఘన
3	Mute చేసిన రకం వెళ్ళకూడదు	ప్రాధాన్యతలు పట్టించుకోవడం లేదు
4	count అసలు events తో సరిపోవాలి	కలపడంలో లెక్క తప్పింది
5	ప్రతి సంఘటనకీ ఒక గమ్యం – పంపాం, కిటికీలో ఉంది, లేదా వేరున్న కారణంతో అణచివేశాం	ఒక notification మౌనంగా పోయింది

నియమం 2 ఒక నిజమైన bug ని పట్టుకుంది

మొదటి పరుగులో 1,200 lo 1,093 విఫలమయ్యాయి: 6 ఆపేసినా పంపాం .

కారణం: నేను opt-out ని అంగీకరించేటప్పుడు చూశాను. కానీ వాడుకరి notification queue lo ఉండగా ఆపేయొచ్చు – మరియు అప్పుడు flush() దాన్ని పంపేస్తుంది.

```
// ముఖ్యం: opt-out ని *ఇక్కడ కూడా* చూడాలి. వాడుకరి notification
// queue lo ఉండగా ఆపేయొచ్చు – అప్పుడు దాన్ని పంపకూడదు.
if (this.#optedOut.has(b.user)) {
    this.stats.suppressedOptOut++;
    this.stats.suppressedEvents += b.events.length;
    continue;
}
```

ఈ BUG ఒక అనుమతి BUG – మరియు అవి అత్యంత ఖరీదైనవి

"Notifications ఆపు" అని నొక్కిన తర్వాత కూడా notification రావడం – అది ఒక తాంత్రిక లోపం కాదు, అది ఒక సమ్మకద్రోహం. మరియు చాలా చోట్ల అది చట్టవిరుద్ధం కూడా (GDPR, CAN-SPAM).

మరియు ఈ bug యొక్క ఆకారం చాలా సాధారణమైనది: ప్రవేశ ద్వారం దగ్గర పరీక్షించి, నిష్క్రమణ ద్వారం దగ్గర మర్చిపోవడం. కిటికీ ఉన్న ప్రతి system lo ఇది జరగగలదు – ఎందుకంటే ప్రవేశానికి నిష్క్రమణకీ మధ్య సమయం ఉంది, మరియు ఆ సమయంలో ప్రపంచం మారుతుంది.

1,200 యాదృచ్ఛిక ప్రయోగాలు · 7,46,429 సంఘటనలు
నియమ ఉల్లంఘనలు: 0

ఏ దారులు నడిచాయి:

received	7,46,429
coalesced	18,376
sent	57,283
suppressedCap	21,225
suppressedOptOut	6,42,302
suppressedPref	7,209
deferredQuiet	1,41,459
suppressedEvents	32,554

Test విఫలం కాగలదా? – ఐదు మార్పులు

మార్పు లేని code → 0/1200 విఫలం

రేజిస్టర్ పరిమితి పరీక్ష తీసేస్తే → 313/400 విఫలం

ఉదా: 0: ఒక రేజిస్టర్ 16 పంపాం · పరిమితి 15

opt-out ని పంపేటప్పుడు చూడకపోతే → 367/400 విఫలం

ఉదా: 6 ఆపేసినా పంపాం

mute పరీక్ష తీసేస్తే → 288/400 విఫలం

ఉదా: 6 mute చేసిన comment ని అంగీకరించాం

నిశ్శబ్ద గంటల్లో వాయిదా కాకుండా పారేస్తే → 371/400 విఫలం

ఉదా: లెక్క తప్పింది: అంగీకరించినవి 120 ≠ 99

కలపకుండా ప్రతిదీ వెంటనే పంపితే → 0/400 విఫలం

నాలుగో మార్పు – నా నియమం ముందు చాలా బలహీనంగా ఉంది

"నిశ్శబ్ద గంటల్లో వాయిదా వేయకుండా పారేస్తే" – ఇది ఒక నిజమైన data loss. కానీ నా మొదటి నియమం 5 పంపినవి + పెండింగ్ ≤ అంగీకరించినవి అని చెప్పింది, మరియు పారేసినా అది నిజమే.

దాన్ని సమానత్వంగా మార్చాక – పంపినవి + పెండింగ్ + అణచినవి = అంగీకరించినవి – మరియు అణచివేసినవాటిని batches కాకుండా events గా లెక్కపెట్టాక – అది **371/400** పట్టుకుంది.

"ఎక్కువ కాకూడదు" అనే నియమం కంటే "సరిగ్గా ఇంతే ఉండాలి" అనే నియమం చాలా బలమైనది.

ఐదో మార్పు (కలపడం తీసేయడం) **0/400** – మరియు అది సరైనదే. కలపడం ఒక భారపు లక్షణం, సరి-తప్పు లక్షణం కాదు. దాని రుజువు **54 పట్టిక**, fuzz కాదు.

Part 6 – Interview Io

12. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

నిమిషాలు	ఏమి చెప్పాలి
0-5	"ఇది అసలు పంపాలా?" అనే ప్రశ్నతో మొదలుపెట్టండి. అది మిగతా అందరి కంటే వేరుగా ఉంటుంది
5-9	§2 ప్రశ్నలు. ముఖ్యంగా "వాడుకరి ఆపేస్తే మనకి ఎంత నష్టం?"
9-14	Queue, workers, providers – త్వరగా. ఇది సులభమైన భాగం
14-22	మొదటి విరుపు. వైరల్ post → 50,000 . కలపడం → 98.2% . కానీ OTP కలపకూడదు
22-32	రెండో విరుపు. 243/రీజ. పరిమితి లేకపోతే 78.4% పోతారు. పరిమితి పెట్టి 15.7% ఎక్కువ పంపాం
32-40	మూడో విరుపు. SMS 5% traffic, push p99 16.8 s . Bulkhead → 50 ms
40-45	ఐదు నియమాలు, మరియు opt-out bug

ఏమి తప్పక చెప్పాలి

- సంఘటనలు మరియు **notifications** ఒకటి కాదు. 50,000 likes = 1 notification.
- తక్కువ పంపితే ఎక్కువ చేరుతుంది. పరిమితి 30 → 15.7% ఎక్కువ delivered.
- సహనం ఒక పరిమిత వనరు, మరియు అది తిరిగి నిండదు.
- ఒక queue యొక్క వేగాన్ని దాని అతి నెమ్మదైన వినియోగదారు నిర్ణయించకూడదు.
- ప్రవేశం దగ్గర పరీక్షించడం సరిపోదు – కిటికీ ఉన్న చోట నిష్క్రమణ దగ్గర కూడా పరీక్షించాలి.

ఏమి వదిలేయాలి

- Retry/backoff/DLQ వివరాలు – "jitter తో exponential backoff" అని ఒక వాక్యం (LLD Deep 25)
- APNS/FCM protocol వివరాలు
- Template rendering, localisation – §14 lo ఒక వాక్యం
- Notification inbox / history UI

13. నీటితో చెప్పాల్సిన English script

"Most systems want more throughput. This one doesn't, and I want to start there. A notification system's real question isn't 'how do I deliver this' – it's '**should I send this at all?**' Because over-sending here doesn't degrade the system, it **permanently loses the user.**"

"The transport is the easy part: a queue, workers, three provider SDKs. I'd spend two minutes on it. The hard parts are three questions: send it **now?** send it **at all?** send it **how?**"

"First, coalescing. A post goes viral – 50,000 likes in four hours. My naive design sends 50,000 notifications, which is **208 per minute**, one every 0.3 seconds. That's not a bug; the code did exactly what I asked. But the user doesn't want 50,000 messages, they want one sentence: '50,000 people liked your post.' **Events and notifications aren't the same thing.** Coalescing into a 30-second window cuts it **98.2%**, to 912; a five-minute window gives 129. This is the rare change that reduces cost *and* improves quality. The caveat is that the window is **per notification type** – OTPs and security alerts must never coalesce."

"Second, and this is the result I'd lead with. A normal user gets **243 notifications a day** – one every six minutes – and no single team did anything wrong; each one is individually reasonable. I modelled tolerance as a finite resource: past thirty a day, each extra notification carries a small chance of the user switching notifications off entirely and permanently. Over thirty days with **no cap, 78.4% of users opt out** – you can reach 107,000 of your 500,000. With a cap of thirty a day, **nobody opts out**, and you deliver **399.7 million notifications instead of 345.6 million – 15.7% more. You send fewer and reach more.** I'd be explicit that the tolerance numbers are assumptions you'd take from your own data; what isn't an assumption is the shape – tolerance is finite and doesn't refill."

"Third, channels. Push is 50 milliseconds, email 400, SMS two seconds – and the SMS provider only accepts ten a second. With one shared queue, push p50 was **7.7 seconds and p99 was 16.8**, because SMS at **5% of volume** was holding all forty workers while it waited on its provider. Separate pools per channel put push at a flat **50 milliseconds** – 336× better. SMS gets worse, 19 seconds to 130, and I'd say that plainly – but the SMS queue was always going to drain slowly, because the provider caps it. **The question isn't whether SMS is fast, it's whether SMS gets to impose its slowness on push.**"

"Five invariants, 1,200 randomized runs, 746,000 events. The fuzz failed **1,093 of 1,200** on the first run with a real bug: I checked opt-out when **accepting** an event, but a user can opt out while their notification is still sitting in the coalescing window, and then flush sends it. That's a consent bug, and in many jurisdictions illegal. The shape is worth remembering – **check at the entrance, forget at the exit** – and it can happen in any system with a delay between the two."

"One more on invariant quality. My accounting rule was originally 'sent plus pending is at most accepted', and a mutation that **drops** deferred quiet-hours notifications passed it – dropping keeps the inequality true. Rewriting it as exact equality, and counting suppressed **events** rather than **batches**, caught it 371 times in 400. **'Exactly this' is a much stronger rule than 'no more than this.'**"

14. Follow-ups – templates, A/B, ఖర్చు

"Templates మరియు localisation ఎక్కడ?"

Notification యొక్క **content** ని pipeline lo చివరి క్షణం వరకు render చేయకూడదు. కారణం §5: కలపడం జరిగేది `user|kind` మీద, మరియు render అయిన తర్వాత "50 మంది like చేశారు" అని రాయడం కష్టం – మీ దగ్గర 50 render అయిన strings ఉంటాయి. కాబట్టి queue lo **structured data** ఉండాలి (`{ kind, actors[], target }`), మరియు render పంపేటప్పుడు జరగాలి. అప్పుడు localisation కూడా సులభం – వాడుకరి భాష పంపే క్షణంలో చూస్తాం.

"ఏ notification పనిచేస్తుందో ఎలా తెలుసుకోవాలి?"

ప్రతి notification కి ఒక **ప్రయోగ id** జోడించి, తెరిచిన రేటుని కొలవాలి. కానీ §7 తర్వాత ఒక హెచ్చరిక: **తెరిచిన రేటు ఒక్కటే సరిపోదు**. ఒక notification 5% ఎక్కువ తెరవబడి, opt-out ని 1% పెంచితే, అది **నష్టం** – ఎందుకంటే opt-out శాశ్వతం. కాబట్టి ప్రతి A/B పరీక్షలో **opt-out rate ఒక తప్పనిసరి metric**.

"SMS ఖర్చు తగ్గించడం ఎలా?"

SMS push కంటే వందల రెట్లు ఖరీదు, కాబట్టి **routing ఒక ఖర్చు నిర్ణయం**. నియమం: push ప్రయత్నించు → పరికరం లేకపోతే/విఫలమైతే email → అది కూడా కాకపోతే SMS. మరియు **SMS ని ఒక్కో వాడుకరికి కఠినంగా పరిమితం** చేయాలి (రోజుకి 1-2), ఎందుకంటే §7 యొక్క సహనం SMS కి చాలా తక్కువ.

"ఒక వాడుకరికి 4 పరికరాలు ఉంటే?"

Push ని అన్ని పరికరాలకీ పంపాలి, కానీ **ఒకదానిలో చదివితే మిగతావాటిలో తీసేయాలి** (badge sync). ముఖ్యమైన విషయం: అది **ఒకే notification**, నాలుగు కాదు – కాబట్టి §7 యొక్క రోజువారీ పరిమితి **వాడుకరికి**, పరికరానికి కాదు.

"అత్యవసర notifications (OTP) ఈ pipeline lo ఎలా?"

అవి ఈ pipeline lo ఉండకూడదు. వాటికి సొంత దారి – కలపడం లేదు, రోజువారీ పరిమితి లేదు, నిశ్శబ్ద గంటలు లేవు, మరియు సొంత worker pool (§10). ఎందుకంటే అవి notifications కాదు – అవి ఒక **synchronous flow యొక్క భాగం**, వాడుకరి వాటి కోసం తెర చూస్తూ ఎదురుచూస్తున్నాడు.

15. ఏమి నేర్చుకున్నాం

- 1. సంఘటనలు మరియు notifications ఒకటి కాదు.** 50,000 likes అంటే 50,000 సందేశాలు కాదు – అది ఒక్క వాక్యం. ఆ రెండింటినీ ఒకటే అనుకోవడం ప్రతి notification system యొక్క మొదటి తప్పు.
- 2. తక్కువ పంపితే ఎక్కువ చేరుతుంది.** పరిమితి లేకుండా 345.6 మిలియన్ పంపి 78.4% వాడుకరులని పోగొట్టుకున్నాం. రోజుకి 30 పరిమితితో **399.7 మిలియన్ పంపి ఒక్కరినీ** పోగొట్టుకోలేదు.
- 3. సహనం ఒక పరిమిత వనరు, మరియు అది తిరిగి నిండదు.** Database పడిపోతే లేస్తుంది. Notifications ఆపేసిన వాడుకరి తిరిగి రాడు.
- 4. ఒక queue యొక్క వేగాన్ని దాని అతి నెమ్మదైన వినియోగదారు నిర్ణయించకూడదు.** SMS 5% traffic, కానీ push p99 ని **16.8 సెకన్లు** చేసింది. Bulkhead తో **50 ms**.
- 5. ప్రవేశం దగ్గర పరీక్షించడం సరిపోదు.** నా opt-out పరీక్ష ప్రవేశం దగ్గర ఉంది; వాడుకరి కిటికీలో ఉండగా ఆపేస్తే notification వెళ్ళిపోయింది – **1,200 lo 1,093 సార్లు**. కిటికీ ఉన్న ప్రతి చోటా ఈ ప్రమాదం ఉంది.
- 6. "ఎక్కువ కాకూడదు" కంటే "సరిగ్గా ఇంతే" చాలా బలమైన నియమం.** నా అసమానత నియమం data loss ని దాటేసింది; సమానత్వ నియమం దాన్ని **371/400** పట్టుకుంది.
- 7. వాయిదా వేయడం మరియు పారేయడం ఒకటి కాదు.** నిశ్శబ్ద గంటల్లో notification **ఆగాలి**, పోకూడదు. ఆ రెండింటి మధ్య తేడా code lo ఒక్క పంక్తి, కానీ వాడుకరికి అది ఒక పోయిన సందేశం.

ఈ **DOC** నుంచి ఒక్క వాక్యం గుర్తుపెట్టుకోవాలంటే

ఇది ఒక్కటే system, దీనిలో "ఎక్కువ చేయడం" అనేది ఒక వైఫల్యం.

Database ఎక్కువ queries మోస్తే మంచిది. Cache ఎక్కువ hits ఇస్తే మంచిది. **Notification system ఎక్కువ పంపితే – అది తనని తాను నాశనం చేసుకుంటుంది.**

అందుకే ఈ doc lo ప్రతి పరిష్కారం ఒకే దిశలో ఉంది: **కలుపు (§5), పరిమితింపు (§7), వేరు చేయి (§10)**. మూడూ "తక్కువ, కానీ సరైనది" అనే ఒకే ఆలోచన.